

Agent Foundry

Implementation Guide

Agent Foundry

September 3, 2026

CONTENTS

Agent Foundry	
Implementation Guide	
1. Two Paths In	
2. Fastest Path: the Scaffold Generator	
3. Choosing Your Architecture	
4. Building Your First Agent, Step by Step	
Step 1 — Identity and policy	
Step 2 — Tools	
Step 3 — The prompt	
Step 4 — Everything else, and the graph	
5. Handling Destructive Actions	
6. Configuring Evaluation and Guardrails	
A weighted composite guardrail	
Swapping the guardrail engine itself	
Real policy-as-code	
7. Memory: RAG, a Real Vector Database, and a Knowledge Graph	
8. Multi-Agent Example: Supervisor	
9. Deploying to Production	
As an HTTP API, on any cloud	
As a Slack app	
Real secrets, not environment variables	

Real distributed tracing

10. Customizing Each Layer for Your Use Case

UI/UX Layer

Orchestration Layer

Runtime/Harness Layer

Tools Gateway Layer

LLM Gateway Layer

Context Layer

Guardrails Layer

Security/Observability/Cost Layer

Reasoning/Planning Layer

11. Operational Tooling

Feature flags — ship a change dark, then roll it out

Prompt versioning — publish, and roll back instantly if it regresses

A/B testing a prompt or policy change

Escalating instead of denying

Bulk/offline jobs — batch and scheduling

Cedar as a second policy-as-code option

12. Going-to-Production Checklist

Agent Foundry

Implementation Guide

How to build your agent on top of the reference architecture

1. Two Paths In

Agent Foundry gives you two starting points. Pick based on how much governance your use case needs on day one — you can move from the first to the second later without rewriting your tools.

Quickstart — real `langchain.agents.create_agent`, plain Python functions as tools, native structured tool-calling. Three lines of code, no RBAC, no guardrails, no eval. The on-ramp.

```
from agent_foundry.quickstart import plug_and_play_agent
from langchain_anthropic import ChatAnthropic

def lookup_order(order_id: str) -> str:
    """Look up the status of an order by its id."""
    return db.get(order_id)

agent = plug_and_play_agent(
    ChatAnthropic(model="claude-sonnet-5"),
    tools=[lookup_order],
    system_prompt="You are a support agent.",
)
agent.invoke({"messages": [{"role": "user", "content": "status of order A100?"}]}, config)
```

Governed — `AgentConfig` plus one of the orchestration topologies. RBAC, guardrails, evaluation, cost tracking, audit, autonomy levels, multi-agent patterns. This is the rest of this guide.

Tools written for one path work in the other: `quickstart.to_langchain_tool(s)()` converts a governed `ToolSpec` into a real `LangChain` tool.

2. Fastest Path: the Scaffold Generator

For the governed path, don't hand-wire the files. Generate them:

```
python -m agent_foundry.scaffold sales_agent --tools  
lookup_lead,send_email
```

This writes two files:

- `prompts/sales_agent.md` — a starter prompt with your tool list already listed, marked `TODO: describe this agent's job, tone, and what it should refuse to do`
- `agents/sales_agent.py` — a complete, runnable script: identity, tool registry with stub functions raising `NotImplementedError`, a policy, and a compiled graph, wired to a CLI chat loop

Fill in the TODOs — the tool function bodies and the prompt content — set `ANTHROPIC_API_KEY`, and run it. Everything else (the gateways, the guardrail engine, the eval harness, the runtime budget, the tracer) is already correctly wired; this is what actually moves build time from days to hours. The architecture makes an agent *possible* to build quickly; the scaffold makes it *fast to start*.

3. Choosing Your Architecture

Before writing code, answer one question: **how many agents does this task actually need, and how do they need to relate to each other?**

Your situation	Use	Why
One agent, a set of tools	<code>build_agent_graph</code>	The default. Start here unless you have a specific reason not to.
Several distinct specialists, one entry point that routes	<code>build_supervisor_graph</code>	Support triage that routes to billing / shipping / returns
Agents that hand off directly, no central dispatcher	<code>build_swarm_graph</code>	A researcher agent that pulls in a coder agent mid-task
One job, many inputs, run concurrently	<code>build_fanout_graph</code>	Classify 500 tickets at once
Agents reasoning collaboratively over shared state	<code>build_blackboard_graph</code>	Multi-perspective analysis (a researcher and a skeptic building a shared case)
A decision that benefits from independent takes	<code>build_debate_graph</code>	High-stakes judgment calls, reviewed by a judge agent
A reliable pipeline, no judgment needed at each step	<code>build_dag_graph</code>	fetch → validate → transform → notify
One agent needs another as a single capability	<code>agent_as_tool</code>	A manager agent that calls a research agent once and keeps control
The agent should react to system events, not just chat	<code>events.wire_event_driven</code>	Any topology above, triggered by a message queue or webhook

If you don't want to choose by hand every time, score the request and let a rule set decide:

```
from agent_foundry.kpi import KPIBoard, complexity_kpi, risk_kpi
from agent_foundry.planning import StrategyRule, StrategySelector

board = KPIBoard()
board.register(complexity_kpi())
board.register(risk_kpi())

selector = StrategySelector(
    rules=[
        StrategyRule("single_agent", lambda ctx: ctx["complexity"] <
0.3),
        StrategyRule("debate_judge", lambda ctx: ctx["risk"] > 0.8),
    ],
    default="supervisor",
)
graph = graphs[selector.select({"complexity": score_complexity(request),
"risk": score_risk(request)})]
```

4. Building Your First Agent, Step by Step

This walks through the governed path manually — useful once, so you understand what the scaffold generates for you afterward.

STEP 1 — IDENTITY AND POLICY

```
from agent_foundry.contracts import Identity, Policy, AutonomyLevel

identity = Identity(id="support-agent-1", tenant_id="acme")
policy = Policy(
    allowed_tools=frozenset({"lookup_order", "issue_refund"}),
    max_cost_usd_per_thread=0.50,
    max_steps_per_thread=10,
    requires_approval=frozenset({"issue_refund"}),
    autonomy=AutonomyLevel.L3_APPROVAL,  # the default – see the
autonomy table below
)
```

STEP 2 — TOOLS

Any of these, mixed freely in one `ToolRegistry`:

```

from agent_foundry.contracts import ToolSpec
from agent_foundry.tools_gateway import ToolRegistry
from agent_foundry.mcp_tools import MCPToolSource
from agent_foundry.http_tools import http_tool
from agent_foundry.data_connectors import SQLiteDataSource,
data_query_tool

tools = ToolRegistry()

# a plain function
def lookup_order(order_id: str) -> str:
    return f"order {order_id}: shipped"
tools.register(ToolSpec("lookup_order", "Look up an order by id",
{"order_id": "string"}, lookup_order))

# an MCP server – any server, discovered and registered automatically
mcp = MCPToolSource()
mcp.connect_stdio(command="npx", args=["-y", "@some/mcp-server"])
mcp.register_all(tools)

# a REST API
tools.register(http_tool("order_status", "Look up order status",
url="https://api.example.com/orders/{order_id}"))

# a real database, read-only
db = SQLiteDataSource("orders.db")
tools.register(data_query_tool(db, allowed_tables=frozenset({"orders"})))

```

STEP 3 — THE PROMPT

Write it as a file, not a string literal:

```

prompts/support_agent.md
---
You are a support agent. To use a tool, reply with exactly:
CALL <tool_name> {"arg": "value"}

Tools:
- lookup_order(order_id)
- issue_refund(order_id, amount_usd)

```

```
from agent_foundry.prompts import PromptLibrary
prompts = PromptLibrary("prompts")
system_prompt = prompts.get("support_agent")
```

STEP 4 — EVERYTHING ELSE, AND THE GRAPH

```
from agent_foundry.eval import EvalHarness
from agent_foundry.guardrails import GuardrailEngine
from agent_foundry.llm_gateway import AnthropicProvider, LLMGateway
from agent_foundry.observability import Tracer
from agent_foundry.orchestration import build_agent_graph
from agent_foundry.runtime import RunBudget

thread_id = "demo-1"
graph = build_agent_graph(
    system_prompt=system_prompt,
    llm=LLMGateway(provider=AnthropicProvider()),
    tools=tools,
    guardrails=GuardrailEngine(policy),
    eval_harness=EvalHarness(),
    identity=identity,
    policy=policy,
    budget=RunBudget(policy),
    tracer=Tracer(thread_id),
)

config = {"configurable": {"thread_id": thread_id}}
state = graph.invoke(
    {"messages": [{"role": "user", "content": "status of order A100?"}],
    "thread_id": thread_id},
    config,
)
print(state["messages"][-1]["content"])
```

That's a complete, governed agent: RBAC-checked tool calls, guardrails on input and output, evaluation recorded automatically, a hard cost ceiling, and a full trace.

5. Handling Destructive Actions

Any tool listed in `Policy.requires_approval` pauses the graph instead of executing:

```
from agent_foundry.ui.console import handle_interrupt

state = graph.invoke({"messages": [{"role": "user", "content": "refund
order A100"}]}, config)
state = handle_interrupt(graph, config, state)  # prompts a human,
resumes with their decision
```

`handle_interrupt` is a CLI reference implementation. For a web app or Slack, read `state["__interrupt__"]` yourself and resume with `graph.invoke(Command(resume={"approved": True/False}), config)` from wherever your approval UI lives.

To change *how much* needs a human rather than *which specific tools* do, adjust `Policy.autonomy`:

Set this	To get
<code>L0_ANSWER</code>	The agent can only talk, no tool calls at all
<code>L2_DRAFT</code>	It can prepare non-destructive actions; destructive ones stay blocked outright
<code>L3_APPROVAL</code> (default)	Tools marked <code>requires_approval</code> pause for a human
<code>L4_POLICY_BOUND</code>	It executes autonomously within policy limits, no per-call approval

6. Configuring Evaluation and Guardrails

A WEIGHTED COMPOSITE GUARDRAIL

```
from agent_foundry.kpi import KPIBoard, llm_judge_kpi, db_match_kpi,
reference_check_kpi
from agent_foundry.llm_gateway import make_llm_judge

judge = make_llm_judge(llm, "factual correctness")

board = KPIBoard()
board.register(llm_judge_kpi("correctness", judge=judge, weight=0.5))
board.register(db_match_kpi("order_status_accurate",
    lookup=lambda ctx: (ctx["claimed_status"], db.query(...)[0]
    ["status"]), weight=0.3))
board.register(reference_check_kpi("grounded_in_policy",
    references=lambda ctx: policy_docs, weight=0.2))

score = board.weighted_score(context)      # a single composite number
failing = board.failing(context)           # which KPIs, if any, didn't
meet their threshold
```

Add or remove any KPI at any time with `board.register(...)` / `board.remove(name)`. Write your own with a one-line `KPI(name=..., score=lambda ctx: ..., ...)` for anything domain-specific — an AML flag rate, a first-contact-resolution rate, whatever the use case needs.

SWAPPING THE GUARDRAIL ENGINE ITSELF

`GuardrailEngine` (regex, free) is the default. `LLMGuardrails` (LLM-judgment) catches paraphrased attempts regex misses, at the cost of a model call per check:

```
from agent_foundry.guardrails import LLMGuardrails
guardrails = LLMGuardrails(llm=llm, policy=policy)  # drop-in for
GuardrailEngine
```

REAL POLICY-AS-CODE

If your organization already writes Rego, point at a running OPA server instead of the plain `Policy` dataclass:

```
from agent_foundry.policy_engine import OPAPolicyEngine
engine = OPAPolicyEngine(base_url="http://localhost:8181")
allowed = engine.allow({"tool": "lookup_order", "allowed_tools": [...],
"cost_so_far": 0.01, "max_cost": 1.0})
```

7. Memory: RAG, a Real Vector Database, and a Knowledge Graph

```
from agent_foundry.context import MemoryStore, ChromaVectorStore

memory =
MemoryStore(semantic=ChromaVectorStore(embedding_function=your_embedder))
memory.semantic.upsert(thread_id, "Refunds over $100 require manager
sign-off.", {"source": "policy"})

# knowledge graph with ontology
memory.knowledge_graph.add("order:A100", "placed_by", "customer:42")
memory.knowledge_graph.add_class("gold_customer", parent="customer")
memory.knowledge_graph.instance_of("customer:42", "gold_customer")
memory.knowledge_graph.is_a("gold_customer", "customer") # True,
including transitively

graph = build_agent_graph(..., memory=memory) # RAG is now injected
into the prompt automatically
```

`ChromaVectorStore` accepts any embedding function — OpenAI, Cohere, a local model, or a deterministic offline one for testing. The same `VectorStore` protocol is exactly how Pinecone, Weaviate, Qdrant, or pgvector plug in instead — implement `upsert()` / `search()` and pass it to `MemoryStore`.

8. Multi-Agent Example: Supervisor

```
from agent_foundry.orchestration import AgentConfig,
build_supervisor_graph

billing_config = AgentConfig(system_prompt=prompts.get("billing"),
                             llm=llm, tools=billing_tools,
                             guardrails=GuardrailEngine(billing_policy),
                             eval_harness=EvalHarness(), identity=identity,
                             policy=billing_policy, budget=RunBudget(billing_policy),
                             tracer=Tracer(thread_id))

shipping_config = AgentConfig(system_prompt=prompts.get("shipping"),
                              llm=llm, tools=shipping_tools,
                              guardrails=GuardrailEngine(shipping_policy),
                              eval_harness=EvalHarness(), identity=identity,
                              policy=shipping_policy, budget=RunBudget(shipping_policy),
                              tracer=Tracer(thread_id))

graph = build_supervisor_graph(
    supervisor_prompt="Route the user to the right specialist.",
    agents={"billing": billing_config, "shipping": shipping_config},
    llm=llm,
)
```

Each specialist is a fully independent `AgentConfig` — its own tools, guardrails, policy, and budget. Nothing is shared unless you deliberately pass the same object into more than one config.

9. Deploying to Production

AS AN HTTP API, ON ANY CLOUD

```
from agent_foundry.serve import build_http_app
app = build_http_app(graph)  # POST /chat, GET /health
```

```
docker build -t my-agent .
docker run -p 8080:8080 -e ANTHROPIC_API_KEY=... my-agent
```

The included `Dockerfile` has no cloud-specific code in it — the same image runs on AWS ECS/Fargate/App Runner, GCP Cloud Run, Azure Container Apps, any Kubernetes cluster, or a bare VM.

AS A SLACK APP

```
from agent_foundry.channels import build_slack_app
app = build_slack_app(graph,
    signing_secret=os.environ["SLACK_SIGNING_SECRET"],
    bot_token=os.environ["SLACK_BOT_TOKEN"])
```

Point your Slack app's Event Subscriptions URL at `/slack/events` on wherever you deploy this.

REAL SECRETS, NOT ENVIRONMENT VARIABLES

```
from agent_foundry.security import VaultCredentialProvider
vault = VaultCredentialProvider(base_url="https://vault.internal:8200",
    token=os.environ["VAULT_TOKEN"])
api_key = vault.get("anthropic_api_key")
```

REAL DISTRIBUTED TRACING

```
from agent_foundry.observability import OTelTracer
tracer = OTelTracer(thread_id, processor=your_otlp_span_processor)  #
ships to Jaeger/Honeycomb/Datadog/etc.
```

10. Customizing Each Layer for Your Use Case

Every layer below ships with a sane default. The point of the architecture is that you only touch the layer that actually differs between use cases — everything else stays exactly as scaffolded. This section walks the stack top to bottom with a concrete “what would I actually change here” example for four different agents: a **support agent** (low risk, high volume), a **sales/outreach agent** (external-facing, reputational risk), an **internal research analyst** (high autonomy, low blast radius), and a **fintech compliance agent** (high blast radius, regulated).

UI/UX LAYER

Use case	What changes
Support agent	<code>serve.py</code> 's hosted chat widget as-is — customers expect a chat box
Sales agent	<code>channels.py</code> Slack app instead — reps live in Slack, not a separate tab
Research analyst	CLI (<code>ui/console.py</code>) — the user <i>is</i> the developer
Fintech compliance	Hosted chat, but every <code>requires_approval</code> interrupt renders inline with the source citation, not just “approve/deny”

ORCHESTRATION LAYER

Use case	What changes
Support agent	<code>build_supervisor_graph</code> — one router, specialists for billing/shipping/returns
Sales agent	<code>build_agent_graph</code> — single agent, no need for multiple specialists
Research analyst	<code>build_debate_graph</code> — two independent takes plus a judge, because a wrong research conclusion is expensive to act on
Fintech compliance	<code>build_dag_graph</code> — a fixed pipeline (fetch → check → flag → report), because compliance work should not exercise judgment about <i>what step comes next</i>

RUNTIME/HARNESS LAYER

Use case	What changes
Support agent	<code>max_steps_per_thread=10</code> , <code>LatencyBudget(max_seconds=15)</code> — a slow reply loses the customer
Sales agent	<code>max_cost_usd_per_thread=0.20</code> — outreach is high-volume, keep per-lead cost low
Research analyst	<code>max_steps_per_thread=50</code> , no latency budget — depth matters more than speed
Fintech compliance	<code>CircuitBreaker</code> failure threshold lowered to 1 — one bad tool call halts the run rather than retrying against a regulated system

TOOLS GATEWAY LAYER

Use case	What changes
Support agent	<code>data_query_tool</code> against the orders DB, <code>allowed_tables={"orders"}</code> only
Sales agent	<code>http_tool</code> against the CRM API, plus an MCP connection to the email-sending server
Research analyst	<code>MCPToolSource</code> to a web-search/browsing MCP server, <code>code_execution_tool()</code> for on-the-fly analysis
Fintech compliance	Same <code>data_query_tool</code> , but wrapped with <code>sandbox.run_sandboxed</code> disabled entirely — no arbitrary code execution tool registered at all

LLM GATEWAY LAYER

Use case	What changes
Support agent	Cheap/fast model (<code>task="cheap"</code> routing), <code>PromptCache</code> on — the same policy answers repeat constantly
Sales agent	Mid-tier model, <code>image_content()</code> used to let the agent read a screenshot of a lead's website
Research analyst	Frontier model, <code>.stream()</code> used so the analyst sees reasoning as it's produced
Fintech compliance	Frontier model, cache off — every answer must be freshly grounded, not served from a stale cached response

CONTEXT LAYER

Use case	What changes
Support agent	<code>ChromaVectorStore</code> over the policy/FAQ docs; no knowledge graph needed
Sales agent	Vector store over case studies and pricing sheets, refreshed daily
Research analyst	Full <code>KnowledgeGraph</code> with ontology (<code>add_class</code> / <code>instance_of</code>) — relationships between entities matter, not just similarity
Fintech compliance	Vector store scoped per-tenant (never cross-tenant retrieval), with <code>{"source": ..., "effective_date": ...}</code> metadata so nothing outdated gets cited

GUARDRAILS LAYER

Use case	What changes
Support agent	Default <code>GuardrailEngine</code> (regex) — cheap, and false positives just mean a normal reply
Sales agent	<code>LLMGuardrails</code> on output — a paraphrased overpromise (“we’ll basically guarantee...”) is worse externally than internally
Research analyst	Guardrails mostly off (<code>L4_POLICY_BOUND</code>) — the point is autonomy; correctness is checked via <code>kpi.completeness_kpi</code> , not gated at every step
Fintech compliance	<code>autonomy=AutonomyLevel.L3_APPROVAL</code> , <code>requires_approval</code> on every write action, <code>OPAPolicyEngine</code> in front of the plain <code>Policy</code> for auditable, versioned rules

SECURITY/OBSERVABILITY/COST LAYER

Use case	What changes
Support agent	<code>JSONLAuditLog</code> is enough; <code>CostLedger.by_tenant()</code> reviewed weekly
Sales agent	Same, plus <code>check_alerts</code> wired to page if tool error rate spikes (a broken CRM integration silently failing is worse than it being down)
Research analyst	<code>Tracer</code> reviewed manually per session — this is exploratory, not high-volume
Fintech compliance	<code>VaultCredentialProvider</code> (not env vars) for every credential, <code>OTelTracer</code> shipping every span to a retained compliance log, audit log is the system of record for an examiner

REASONING/PLANNING LAYER

Use case	What changes
Support agent	None — a single ReAct loop is sufficient
Sales agent	<code>BanditSelector</code> over 2-3 outreach message variants, learning which converts
Research analyst	<code>DecisionMaker</code> protocol with a custom cost-aware chooser between “quick answer” and “deep multi-source research”
Fintech compliance	None — planning flexibility is exactly what you don’t want in a regulated pipeline; <code>build_dag_graph</code> ’s fixed order <i>is</i> the control

The shape of the change is always the same: swap one component behind its Protocol (`Provider`, `VectorStore`, `GuardrailChecks`, `DecisionMaker`, `PolicyEngine`, `EventBus`, ...), or adjust one field on `Policy` / `AgentConfig`. Nothing above required touching `orchestration.py`, `runtime.py`, or any other core module — that’s the point of the layering.

11. Operational Tooling

Four utilities that sit alongside the layers above rather than inside one of them — reach for these once an agent is past the prototype stage and into iteration.

FEATURE FLAGS — SHIP A CHANGE DARK, THEN ROLL IT OUT

```
from agent_foundry.feature_flags import StaticFeatureFlagProvider

flags = StaticFeatureFlagProvider({"new_refund_prompt": 20}) # 20%
rollout
if flags.is_enabled("new_refund_prompt", identity=identity):
    system_prompt = prompts.get("support_agent_v2")
else:
    system_prompt = prompts.get("support_agent")
```

The same identity always lands on the same side of a percentage rollout — no flapping between turns.

PROMPT VERSIONING — PUBLISH, AND ROLL BACK INSTANTLY IF IT REGRESSES

```
from agent_foundry.prompts import VersionedPromptLibrary
from agent_foundry.versioning import FileVersionStore

prompts =
VersionedPromptLibrary(store=FileVersionStore("prompt_versions"))
v1 = prompts.publish("support_agent",
open("prompts/support_agent.md").read(), label="initial")
# ... later, a new draft underperforms in eval ...
prompts.rollback("support_agent", version=v1)
```


A/B TESTING A PROMPT OR POLICY CHANGE

```
from agent_foundry.experiments import Experiment, ExperimentTracker

experiment = Experiment(name="refund_tone", variants={"formal": 0.5,
"casual": 0.5})
tracker = ExperimentTracker()

variant = experiment.assign(identity)
system_prompt = prompts.get(f"support_agent_{variant}")
# ... after the turn, score it with an existing KPI and record the result
...
tracker.record("refund_tone", variant, board.weighted_score(context))
print(tracker.summary("refund_tone")) # {"formal": {"mean": 0.91, ...},
"casual": {"mean": 0.84, ...}}
```

ESCALATING INSTEAD OF DENYING

When a guardrail check would otherwise flatly block an action, but the right answer is a human decision rather than a hard stop:

```
from agent_foundry.escalation import QueueEscalator
from agent_foundry.contracts import GuardrailResult

def check_large_refund(amount_usd: float) -> GuardrailResult:
    if amount_usd > 500:
        return GuardrailResult(False, "exceeds auto-approval ceiling",
"action", escalate=True)
        return GuardrailResult(True, stage="action")

result = check_large_refund(750.0)
if result.escalate:
    escalator.escalate(identity=identity, reason=result.reason, context=
{"amount_usd": 750.0})
```

BULK/OFFLINE JOBS — BATCH AND SCHEDULING

```
from agent_foundry.batch import run_batch, IntervalScheduler

# re-score 500 old tickets overnight, not live traffic
report = run_batch(graph, [{"message": t.text, "ticket_id": t.id} for t
in tickets],
    thread_id_fn=lambda item: item["ticket_id"], max_workers=8)
print(report.success_rate)

# or run something on a recurring interval
scheduler = IntervalScheduler()
handle = scheduler.schedule(lambda: run_batch(graph, load_new_tickets()),
every_seconds=3600)
```

CEDAR AS A SECOND POLICY-AS-CODE OPTION

`OPAPolicyEngine` (Rego) and `CedarPolicyEngine` (Cedar, AWS's policy language) both satisfy the same `PolicyEngine` protocol with the identical convenience input shape — pick whichever your org already writes:

```
from agent_foundry.policy_engine import CedarPolicyEngine
engine = CedarPolicyEngine() # embedded, no server — unlike OPA
allowed = engine.allow({"identity_id": "support-agent-1", "tool":
"lookup_order",
    "allowed_tools": ["lookup_order"], "cost_so_far": 0.1, "max_cost":
1.0})
```

12. Going-to-Production Checklist

- ☐ Prompt content reviewed — not just the TODO placeholder
- ☐ `Policy.allowed_tools` is the minimum set the agent actually needs, not everything registered
- ☐ `requires_approval` names every destructive tool; autonomy level matches your actual trust in this agent
- ☐ `max_cost_usd_per_thread` and `max_steps_per_thread` set to real, tested limits
- ☐ At least one KPI board covering correctness, with a threshold that would actually catch a bad response
- ☐ Guardrails tested against a deliberately adversarial input, not just the happy path
- ☐ Tracer wired to somewhere you'll actually look (`OTelTracer` to a real collector, or the dashboard reviewed regularly)
- ☐ Secrets in a real vault, not `.env` in production
- ☐ Container built and health-checked (`GET /health`) before pointing traffic at it
- ☐ A rollback plan — the previous prompt/policy version kept, not overwritten

This guide assumes the architecture described in the companion reference document. Module-by-module detail lives there.